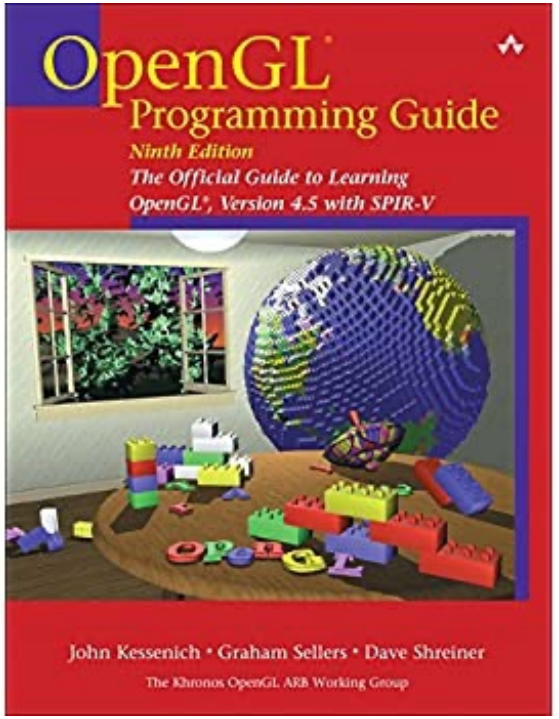
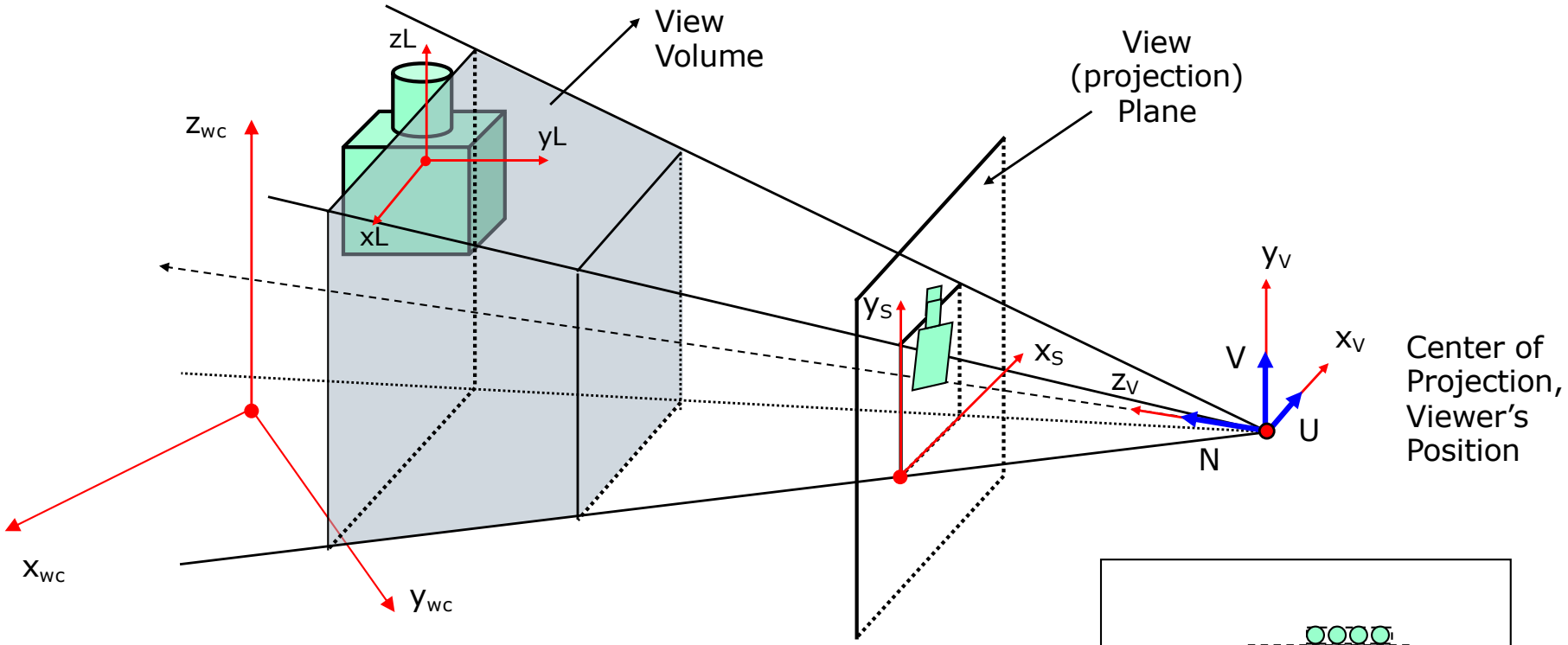
OpenGL – Open Graphics Library

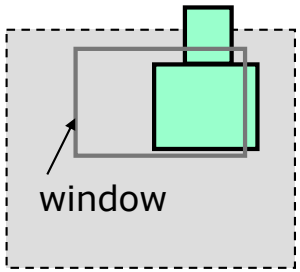
References

1. Shreiner D., Sellers G., Kessenich J., Licea-Kane B., "*OpenGL Programming Guide*", Addison-Wesley, Versions 4.5 with SPIR-V, 9th Edition
2. OpenGL Web site, http://www.opengl.org/
3. Sample code of OpenGL examples, http://www.opengl-redbook.com/

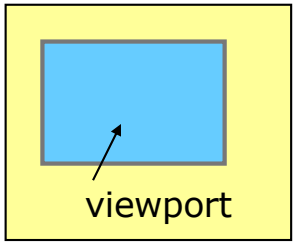


Graphics rendering pipeline

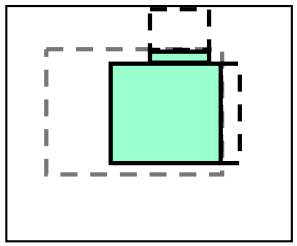




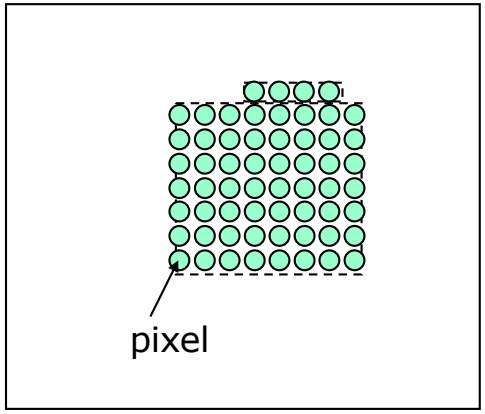
Window definition
in projection plane



Viewport definition
in projection plane

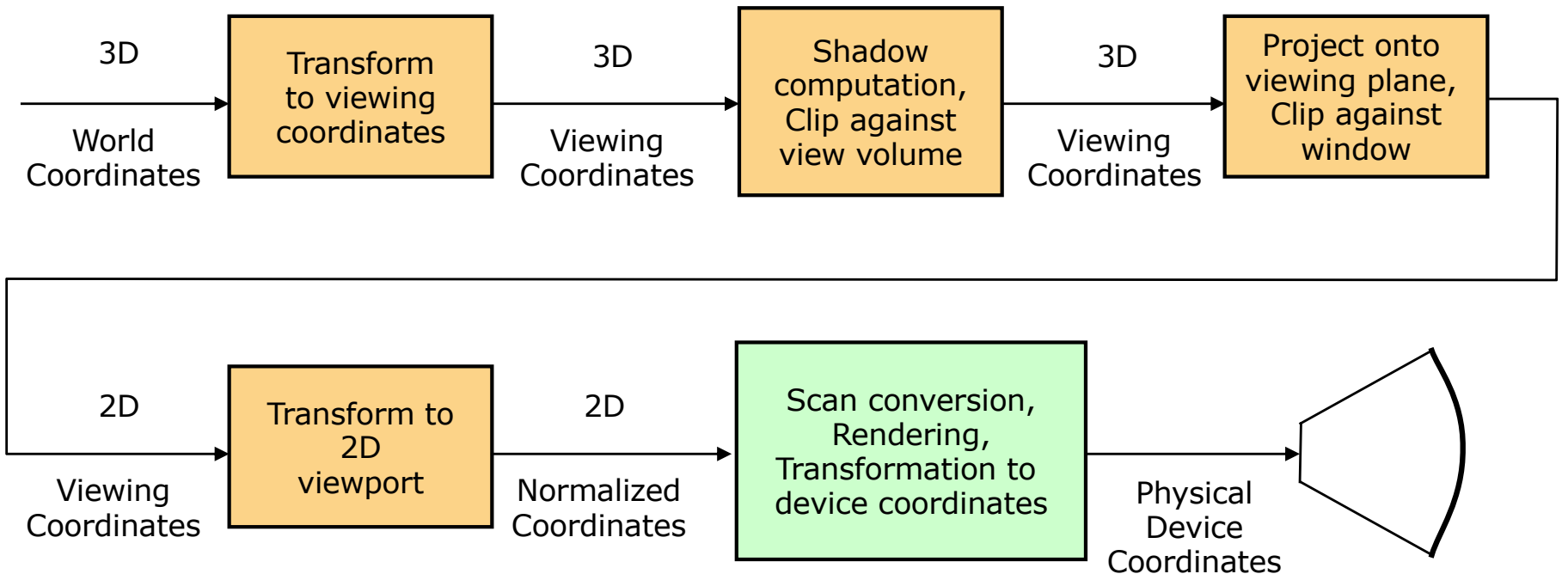


Window's content
inside the viewport



Scan conversion onto the
raster screen

Graphics rendering pipeline



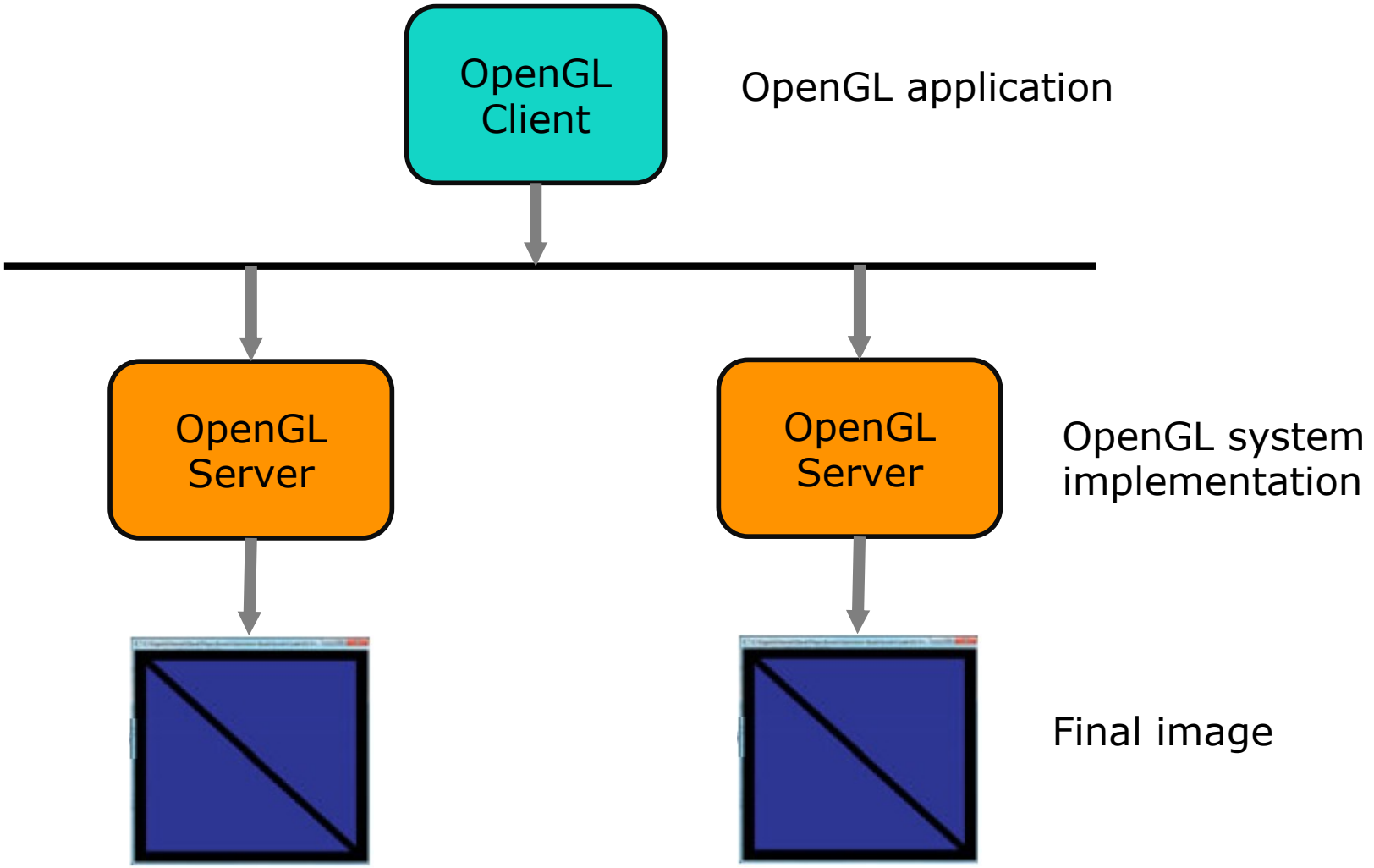
Overview

- Version 1.0 released in January 1992 at Silicon Graphics Computer Systems
- API - application programming interface - software library (~500 distinct commands) for accessing features in graphics hardware
- Designed as a streamlined, hardware-independent interface or entirely in software (if no graphics hardware is present in the system) independent of a computer's operating or windowing system
- It does not include
  - functions for performing windowing tasks
  - functions for processing user input
  - functionality for describing models of three-dimensional objects, or operations for reading image files (e.g., jpeg)

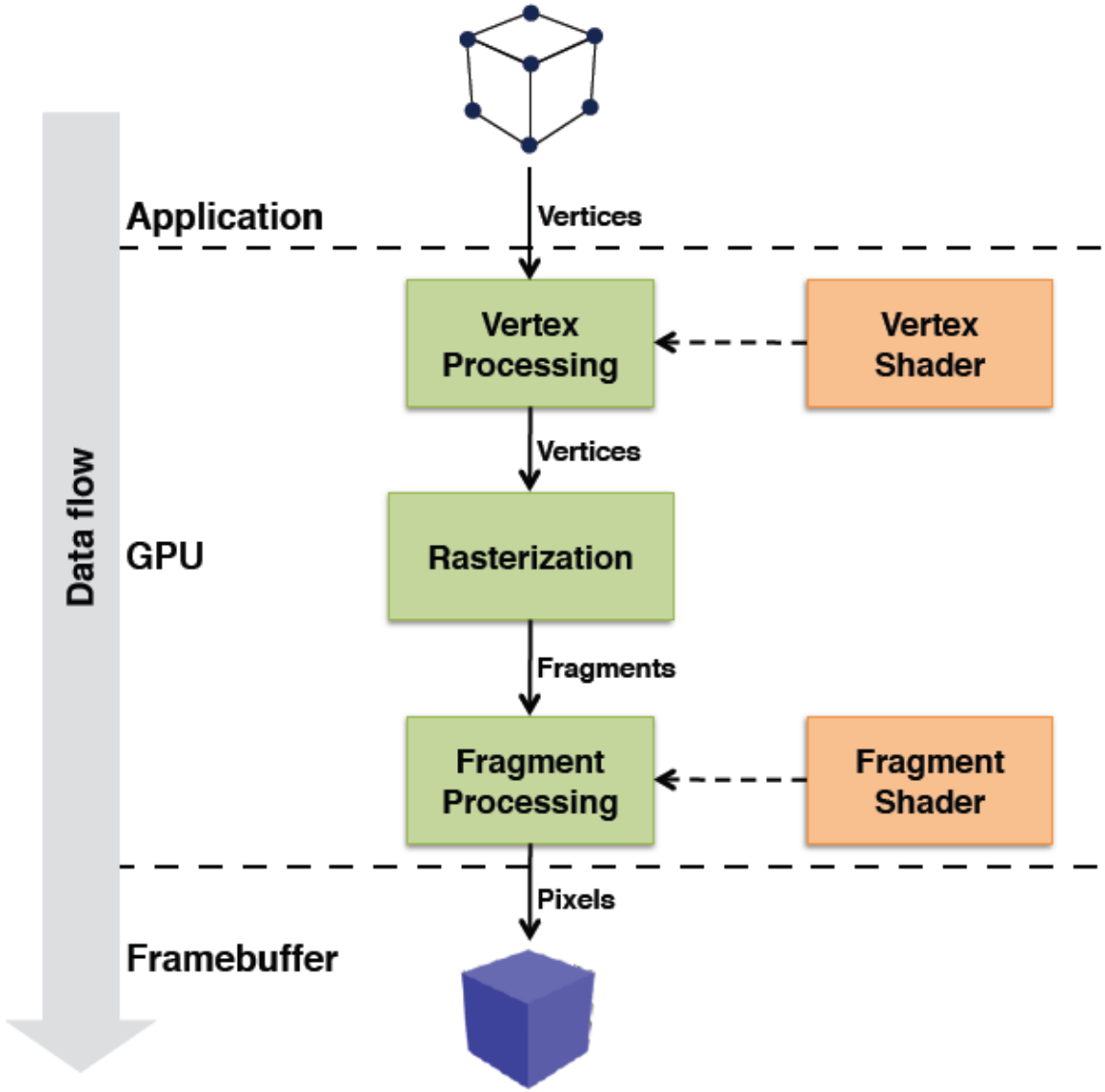
OpenGL system architecture

- Client-server system
- *Client* – OpenGL application developed by programmer
- *Server* - OpenGL implementation provided by the manufacturer of your computer graphics hardware
- In some implementations of OpenGL
    1. Client and server run on the *same machine* (e.g., Mac OS, Windows). The command generated by the client are executed by the server (e.g., GPU modules), which generates the final image within windows.
    2. Client and server run on *different machines* (e.g., X Window System), which are connected by a network. The OpenGL commands are converted into a graphics window system specific protocol that is transmitted to the server via their shared network, where they are executed to produce the final image.

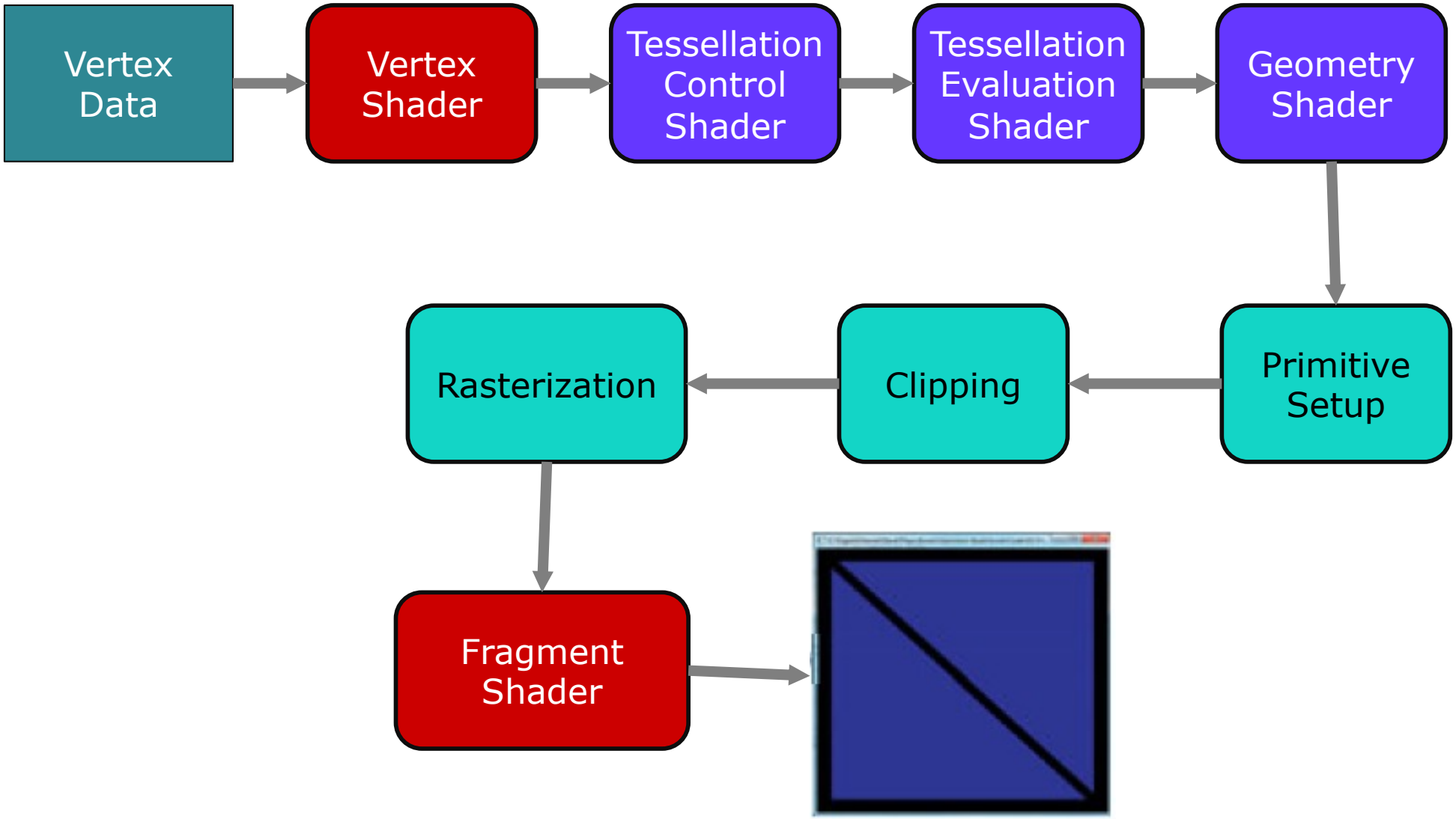
Client – server architecture



Basic graphics pipeline



OpenGL pipeline



OpenGL pipeline description

Main stages:

1. OpenGL pipeline begins with geometric data (vertices and geometric primitives)
2. Processes data through a sequence of shader stages: vertex shading, tessellation shading (control and evaluation), and finally geometry shading
3. The rasterizer generates fragments for any primitive that is inside of the clipping region, and executes a fragment shader for each of the generated fragments

Observation:

- Vertex shaders and fragment shaders must be included
- Tessellation and geometry shaders are optional

Prepare and send data to OpenGL

- Preparing data
  - OpenGL requires that all data be stored in *buffer objects*
  - Buffer object are just chunks of memory managed by the OpenGL server
  - Populating these buffers with data can occur in numerous ways. One of the most common is using the glBufferData() command
- Sending data
  - Drawing in OpenGL means to transfer vertex data to the OpenGL Server
  - Vertex is a bundle of data values that are processed together
  - Data in the vertex bundle can be any data that makes up a vertex (e.g., always includes position data, other data values need to determine the final color of the pixel).

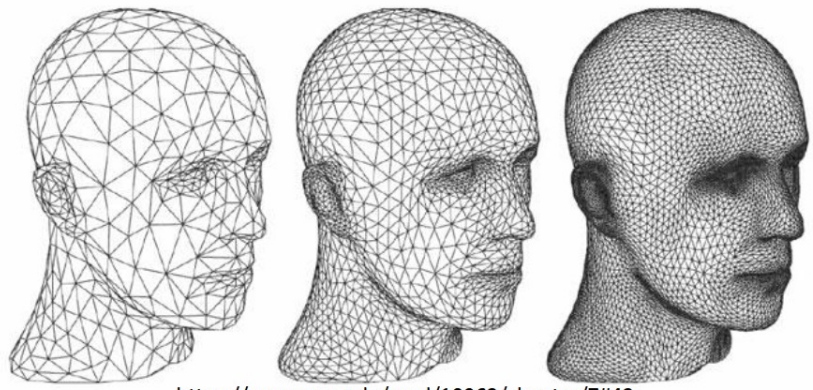
Vertex shading

- Each vertex issued by a drawing command is processed by a vertex shader
- Vertex shaders could have different complexity
  - Very simple - just copying data to pass it through this shading stage
  - Very complex - performing many computations
    - Compute the screen position of the vertex (usually using transformation matrices)
    - Determining the color of the vertex (using lighting computations)
    - Any multitude of other techniques
- An application of any complexity has multiple vertex shaders, but only one can be active at any one time

Tessellation shading

- Tessellation shader stage will continue processing data associated to the vertex, if it has been activated
- Uses *patches* to describe the shape of an object (e.g., triangles, polygons), and allows relatively simple collections of patch geometry to be *tessellated* to increase the number of geometric primitives providing better-looking models (e.g., triangle mesh, polygon mesh)
- The tessellation shading stage can potentially use two shaders to manipulate the patch data and generate the final shape (control and evaluation)

Tessellation



https://www.nap.edu/read/10063/chapter/7#43



https://funnyjunk.com/Math+news/funny-pictures/5656779/27



https://www.geforce.co.uk/whats-new/guides/dx11-guide#1

Geometry shading

- Allows additional processing of individual geometric primitives (e.g., create new primitives before rasterization)
- This shading stage is optional

Geometry shading



http://wesleygamedesigner.blogspot.com/2013/01/developing-particle-system-part-i.html

Primitive assembly

- The previous shading stages all operate on vertices, with the information about how those vertices are organized into geometric primitives being carried along internal to OpenGL
- Primitive assembly stage organizes the vertices into their associated geometric primitives in preparation for clipping and rasterization

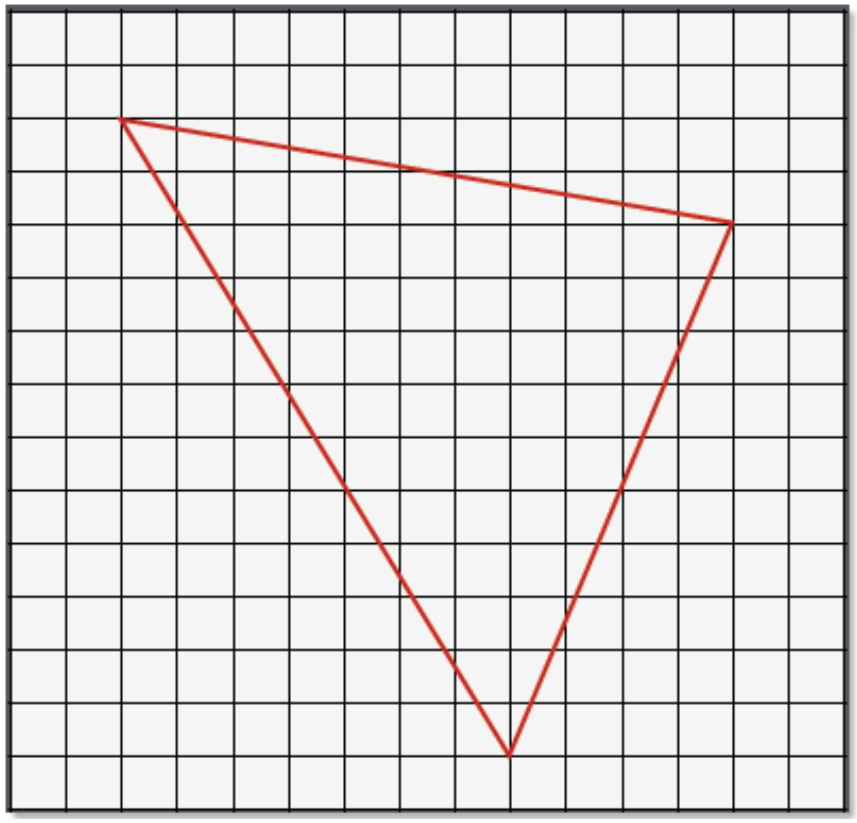
Clipping

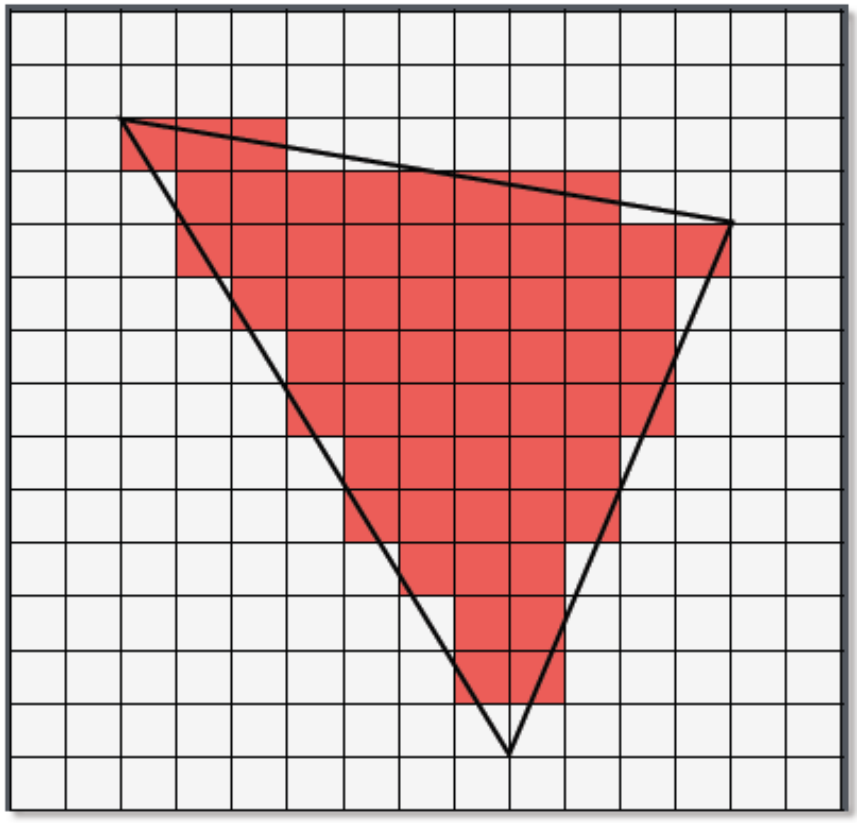
- Primitive associated with the vertex are modified so none of its pixels are outside of the *viewport* (region of the window where you are permitted to draw)
- This operation is called *clipping* and is handled automatically by OpenGL

Rasterization

- After clipping, the updated primitives are sent to the rasterizer for fragment generation
- The fragment is considered a “candidate pixel”. The pixels have positions in the framebuffer, while a fragment still can be rejected and never update its associated pixel location.
- The fragments are processed in the next two stages, fragment shading and per-fragment operations.

Example of rasterization





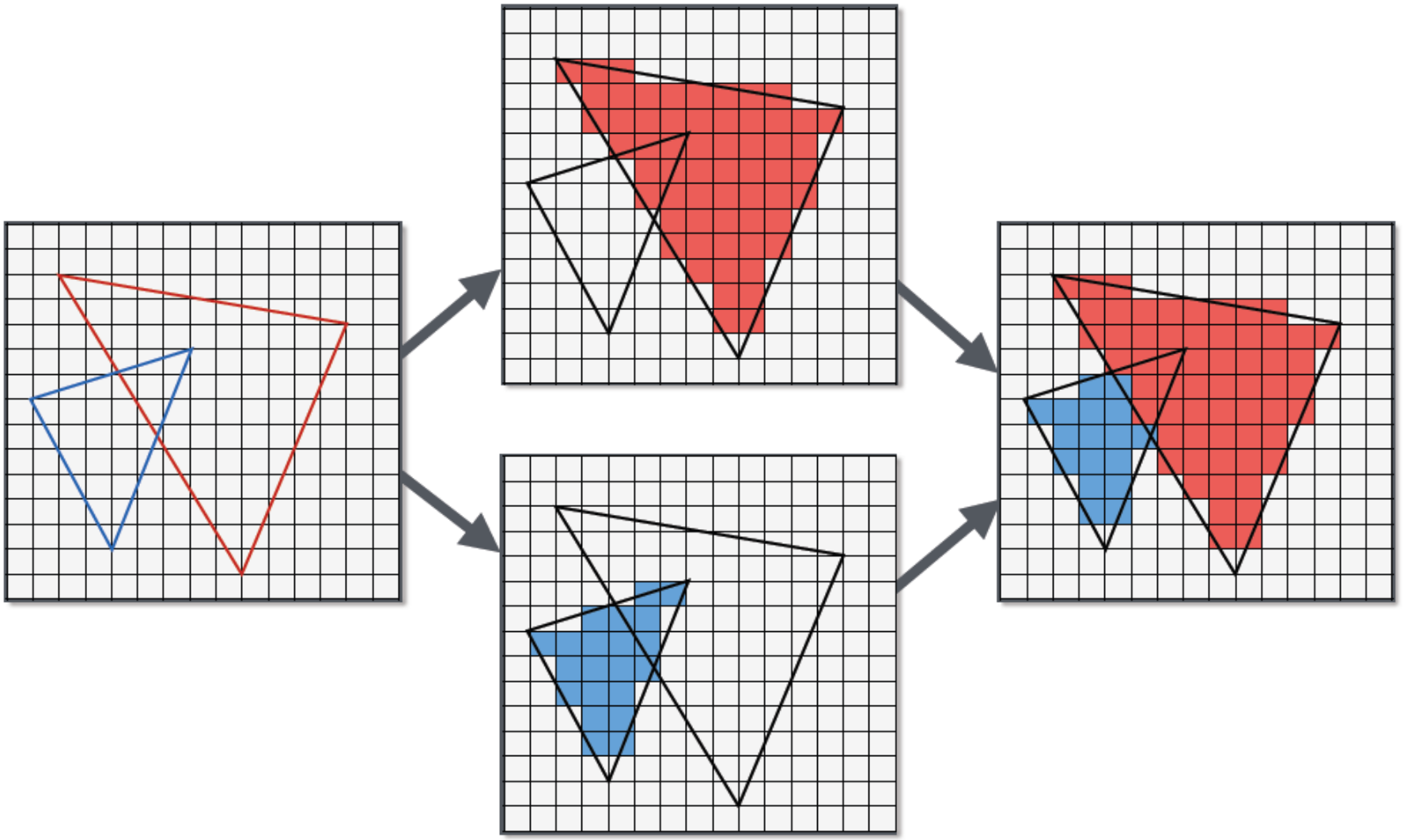
Fragment shading

- It is the final stage where the programmer has control over the color of a screen location (i.e., *fragment color* and *depth value*)
- Fragment shaders are very powerful as they often employ texture mapping to augment the colors provided by the vertex processing stages
- A fragment shader may also terminate processing a fragment if it determines the fragment should not be drawn (fragment *discard process*)
- The final generated image consists of pixels drawn on the screen. A pixel is the smallest visible element on your display. The pixels in your system are stored in a framebuffer, which is a chunk of memory that the graphics hardware manages, and feeds to your display device.

Fragment operations

- Final processing of individual fragments
- Visibility of the fragment is determined using *depth testing* (i.e., *z-buffering*) and *stencil testing*
- If a fragment successfully makes it through all of the enabled tests, then:
  - it may be written directly to the framebuffer, updating the color (and possibly depth value) of its pixel
  - if *blending* is enabled, the color of the fragment is combined with the current color to generate a new color into the framebuffer
- Generally, pixel data comes from an image file, although it may also be created by rending using OpenGL
- Pixel data is usually stored in texture map for use with texture mapping, which allows any texture stage to look up data values from one or more texture maps

Fragment processing



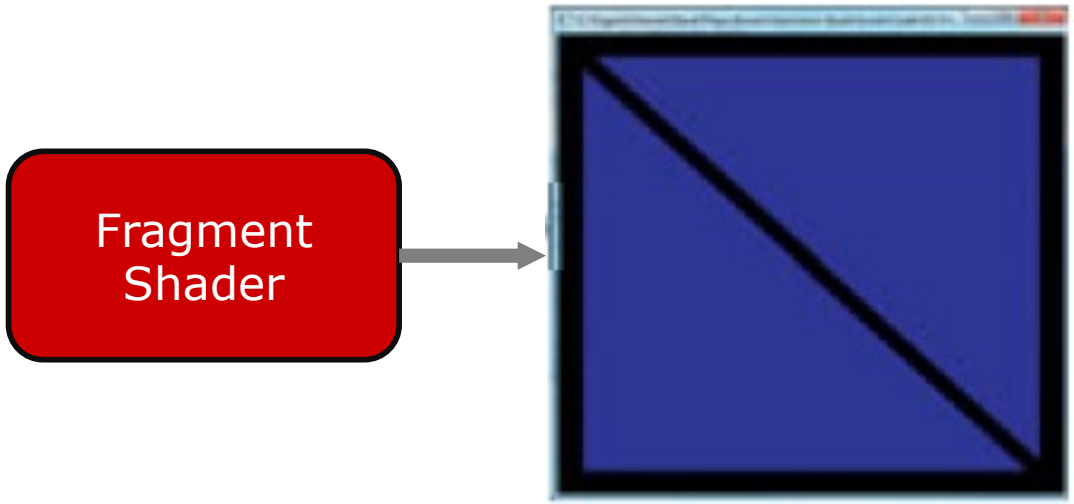
Fragments – lighting, hidden



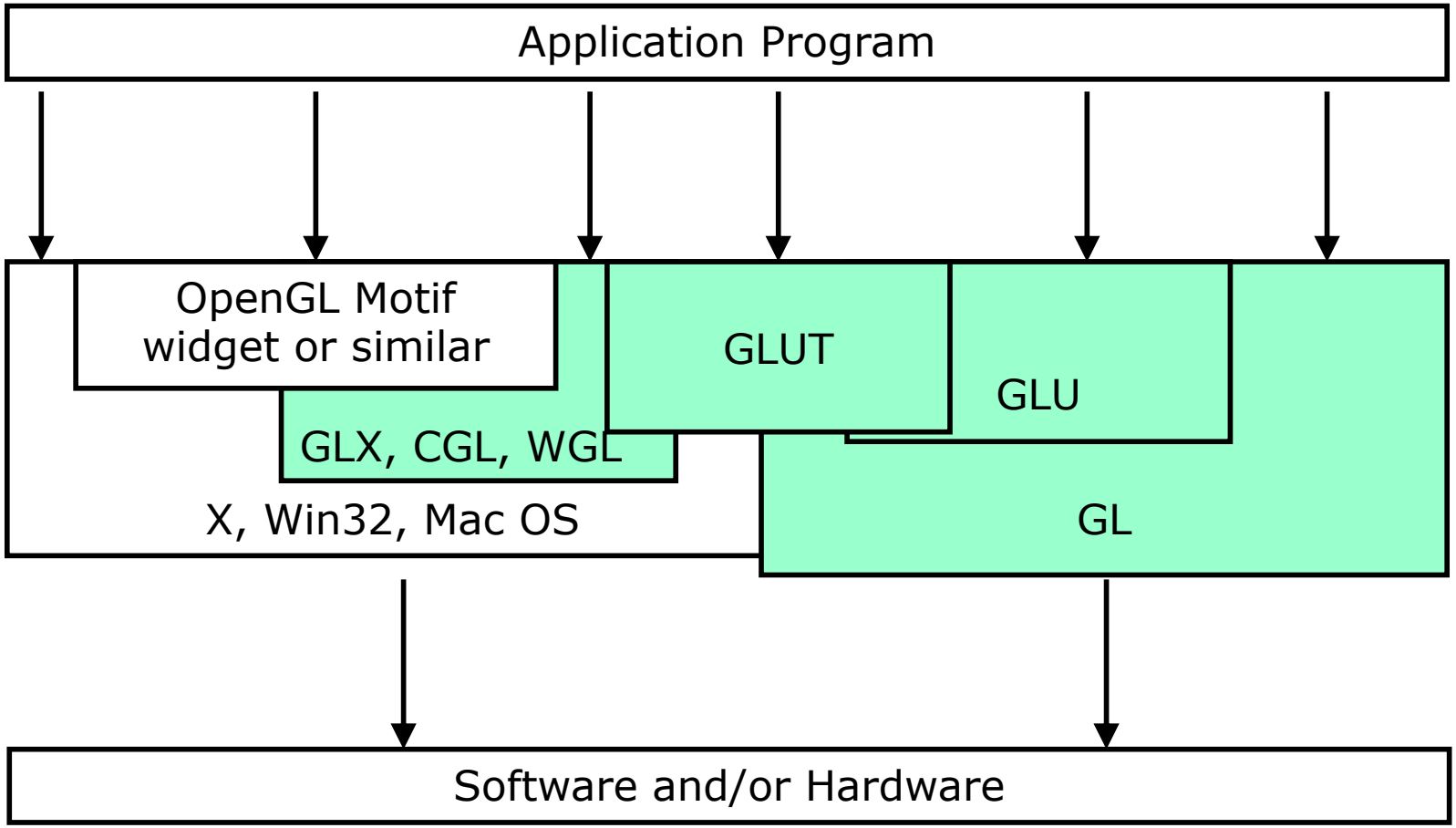


Final output image

- The final generated image consists of pixels drawn on the screen. A *pixel* is the smallest visible element on your display. The pixels in your system are stored in a *framebuffer*, which is a chunk of memory that the graphics hardware manages, and feeds to your display device.

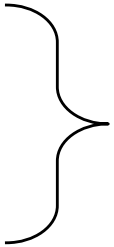


OpenGL programming levels



OpenGL- related libraries

- Window tasks
- User input
- Complex shapes
  - OpenGL Utility Library (GLU)
  - Window system support libraries: GLX / WGL / CGL
  - OpenGL Utility Toolkit (GLUT)
  - OpenGL User Interface (GLUI)
  - OpenInventor



For portability, there are no commands for these

Context and window toolkits

- Creating an OpenGL context is a complex process that varies between operating systems. The automatic OpenGL context creation has become a common feature of several game-development and user-interface libraries, including SDL, Allegro, SFML, FLTK, and Qt
- A few libraries have been designed solely to produce an OpenGL-capable window. The first such library was OpenGL Utility Toolkit (GLUT), later superseded by freeglut

Context and window toolkits

- OpenGL Utility Toolkit (GLUT) – is an old windowing handler, but no longer maintained
- freeglut – is a cross-platform windowing and keyboard-mouse handler; its API is a superset of the GLUT API, and it is more stable and up to date than GLUT
- GLFW is a cross-platform lightweight utility library for use with OpenGL. It provides programmers with the ability to create and manage windows and OpenGL contexts, as well as handle joystick, keyboard and mouse input

OpenGL multimedia libraries

- Several "multimedia libraries" can create OpenGL windows. In addition to input, sound and other tasks useful for game-like applications:
  - Allegro 5 – cross-platform multimedia library with a C API focused on game development
  - Simple DirectMedia Layer (SDL) – cross-platform multimedia library with a C API
  - SFML – cross-platform multimedia library with a C++ API and multiple other bindings to languages such as C#, Java, Haskell, and GOlangWidget toolkits
  - FLTK – small cross-platform C++ widget library
  - Qt – cross-platform C++ widget toolkit. It provides many OpenGL helper objects, which even abstract away the difference between desktop GL and OpenGL

OpenGL supporting languages

- More than ~500 functions/commands similar to those of the programming language C
- Language independent functions
- Language bindings:
  - JavaScript: WebGL (based on OpenGL ES 2.0 for 3D rendering within the web browsers). It uses HTML5 canvas element and is accessed through DOM (Document Object Model) interface.
  - C bindings:
    - WGL (Microsoft Windows interface to OpenGL),
    - GLX (X11 interface to OpenGL)
    - CGL (Mac OS X interface to OpenGL)
  - C binding provided by iOS
  - Java and C bindings provided by Android

Shader languages for GPU

- Assembler
- Cg (NVIDIA)

  C for Graphics
- GLSL (OpenGL)

  OpenGL Shading Language
- HLSL (Microsoft/Direct3D)

  High Level Shading Language

GLSL – OpenGL Shading Language

- GLSL (OpenGL Shading Language), also known as GLslang, is a high level shading language based on the C programming language. It was created by the OpenGL ARB to give developers more direct control of the graphics pipeline without having to use assembly language or hardware-specific languages.
- GLSL is actually two closely related languages. These languages are used to create shaders for the programmable processors contained in the OpenGL processing pipeline.
- The specific languages are referred by the name of the processor they target *vertex* or *fragment*.

GLSL – OpenGL Shading Language

- The *vertex processor* is a programmable unit that operates on incoming vertices and their associated data. Compilation units written in the OGSL to run on this processor are called *vertex shaders*.
- The *fragment processor* is a programmable unit that operates on fragment values and their associated data. Compilation units written in the OGSL to run on this processor are called *fragment* shaders.

Reference:

GLSL specification, http://www.opengl.org/documentation/glsl/

Software technologies

- OS: Windows, Linux, Mac
- API: OpenGL, DirectX, CUDA C/C++, OpenCL , DirectCompute, CUDA Fortran, and others
- Language: Cg, GLSL, HLSL
- Compiler: Cg, OpenGL, DirectX, ASHLI
- Runtime: CgFX, OSG, ASHLI, other codes

OpenGL application program

- Basic structure of an OpenGL application:
  1. *Initialize the state* associated with how objects should be rendered
  2. Specify those objects *to be rendered*

Questions and problems

1. Explain the reason OpenGL is considered an API, library, and graphics system.
2. Where is located the data files (textures, graphics objects) in the client-server architecture of OpenGL? Explain the reasons for such a solution.
3. Explain the difference between fragment and pixel.
4. Define and exemplify the notion of shader.
5. Why only the Vertex and Fragment shaders must be included within the OpenGL pipeline?

Questions and problems

6. Explain the mechanism of transferring data through buffer objects to the OpenGL Server.
7. Explain the differences between the Vertex and Geometry shaders.
8. Exemplify the graphics processing of a cube as 3D object, along the OpenGL pipeline.